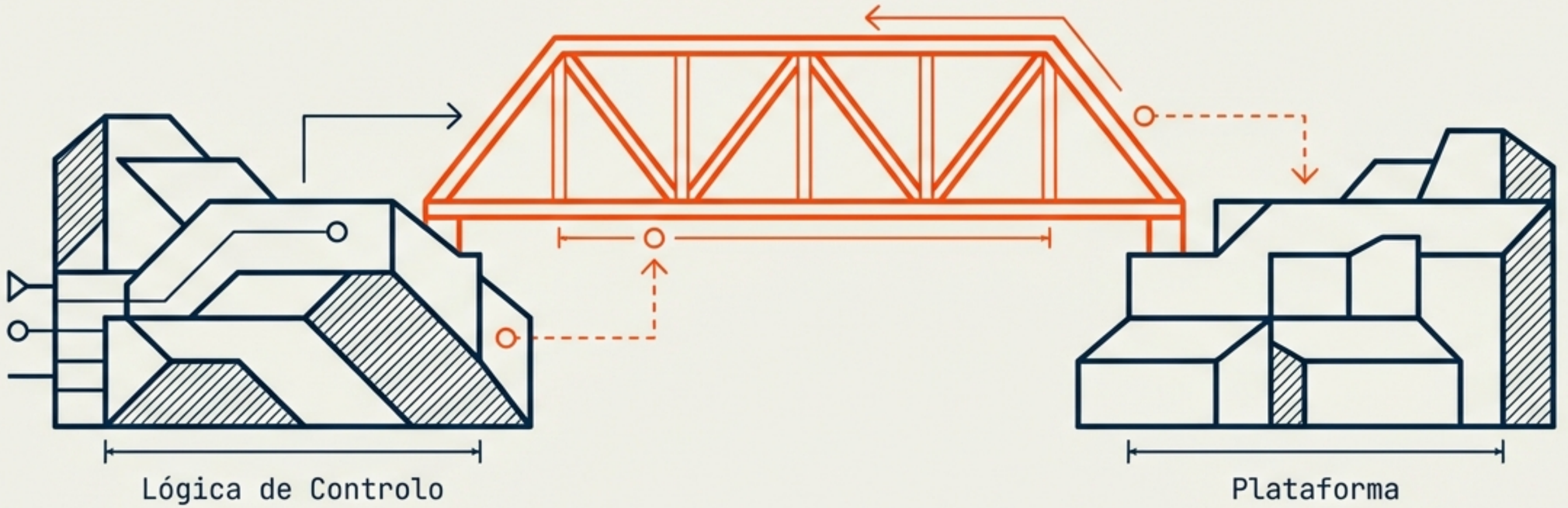
O Padrão Bridge

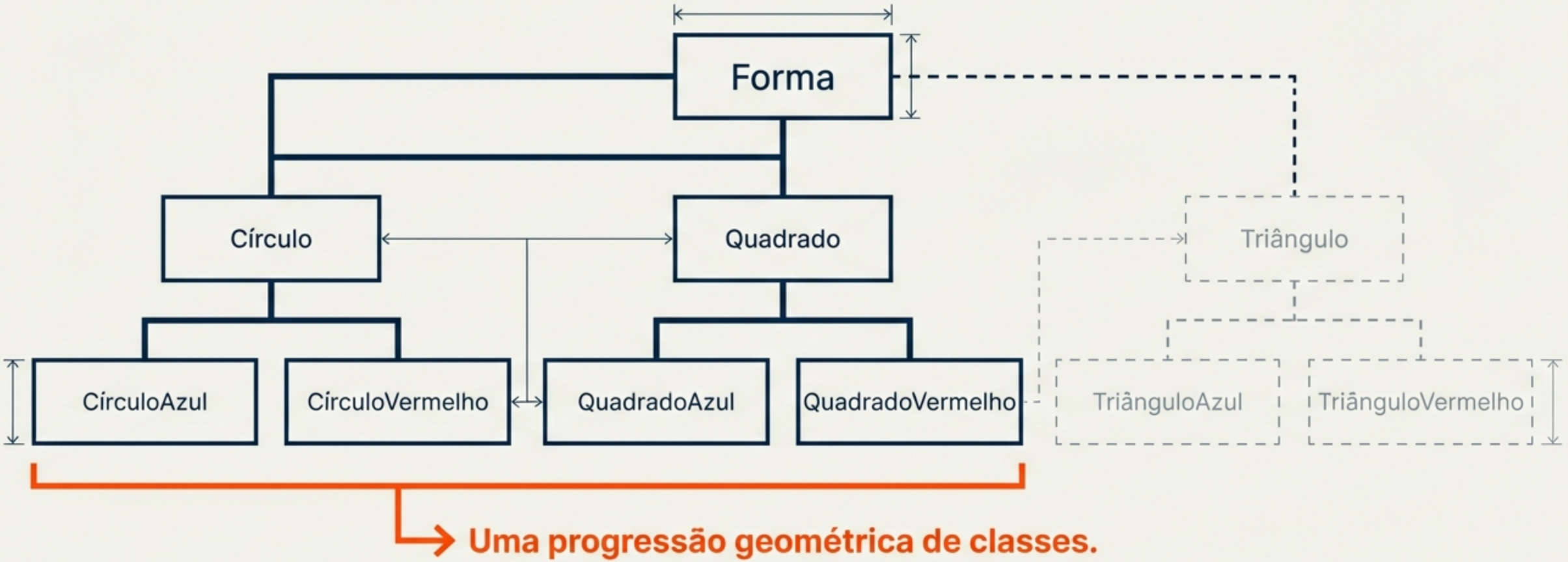
Desacoplar a Abstração da Implementação.



Padrões de Desenho Estruturais

NotebookLM

A Armadilha da Herança Monolítica



Tentar estender classes em duas dimensões independentes (Forma e Cor) resulta numa estrutura rígida e difícil de manter.

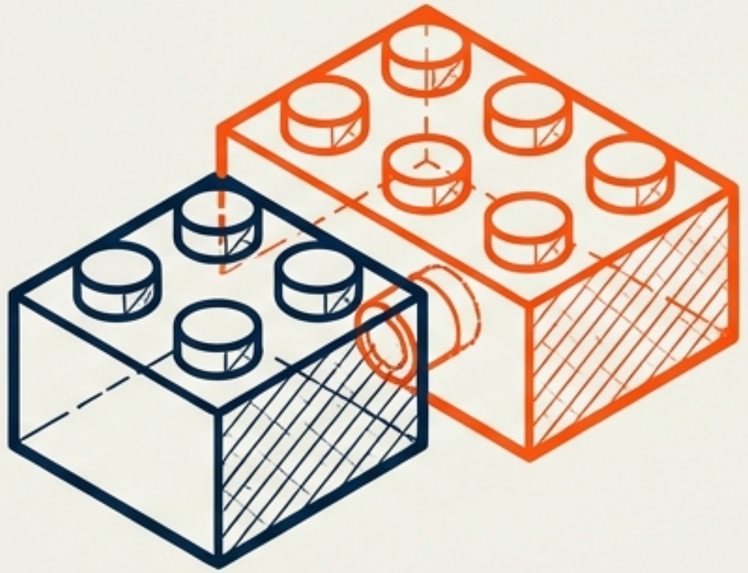
NotebookLM

Herança vs. Composição



Herança

Estático



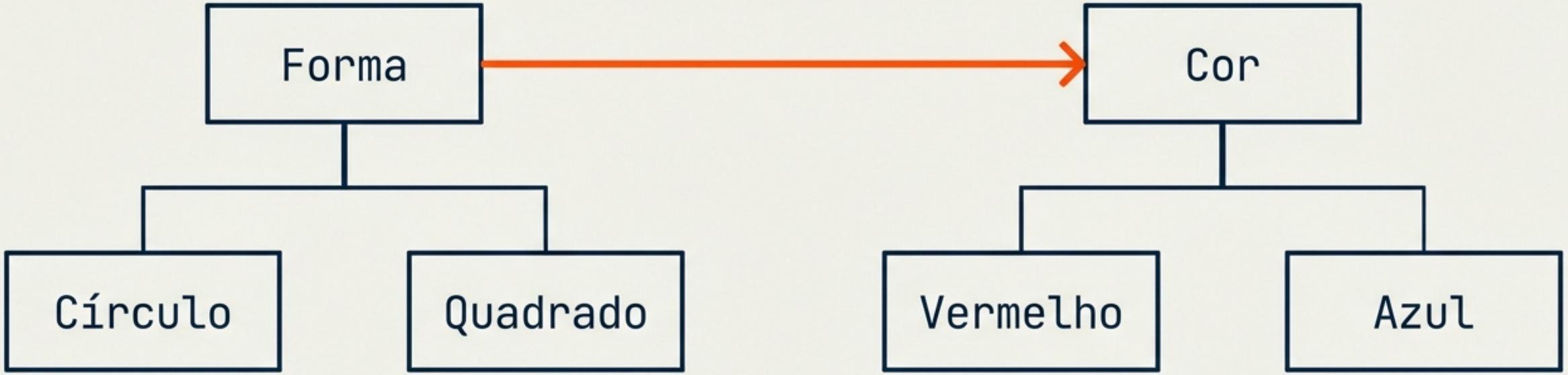
Composição

Dinâmico

A herança liga a implementação à abstração permanentemente. O Padrão Bridge resolve isto substituindo a herança pela composição de objetos.

NotebookLM

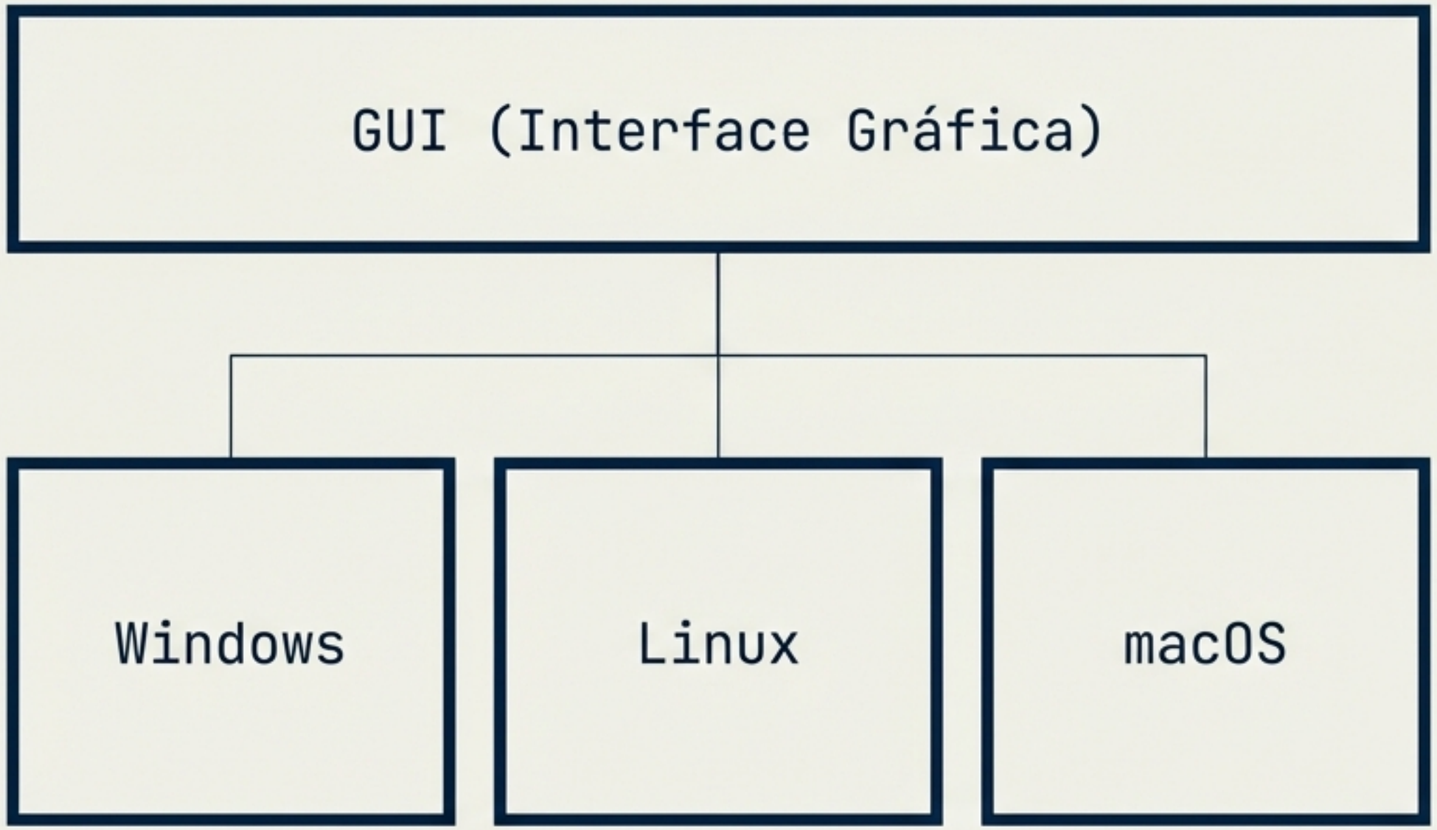
A Mudança de Paradigma



Extraímos uma das dimensões para uma hierarquia separada. Agora, adicionar novas cores não afeta as formas, e vice-versa.

NotebookLM

Descodificar a Terminologia



A Abstração (Interface)

A camada de controlo de alto nível. Não faz o trabalho real; delega-o. (Não é necessariamente uma interface Java).

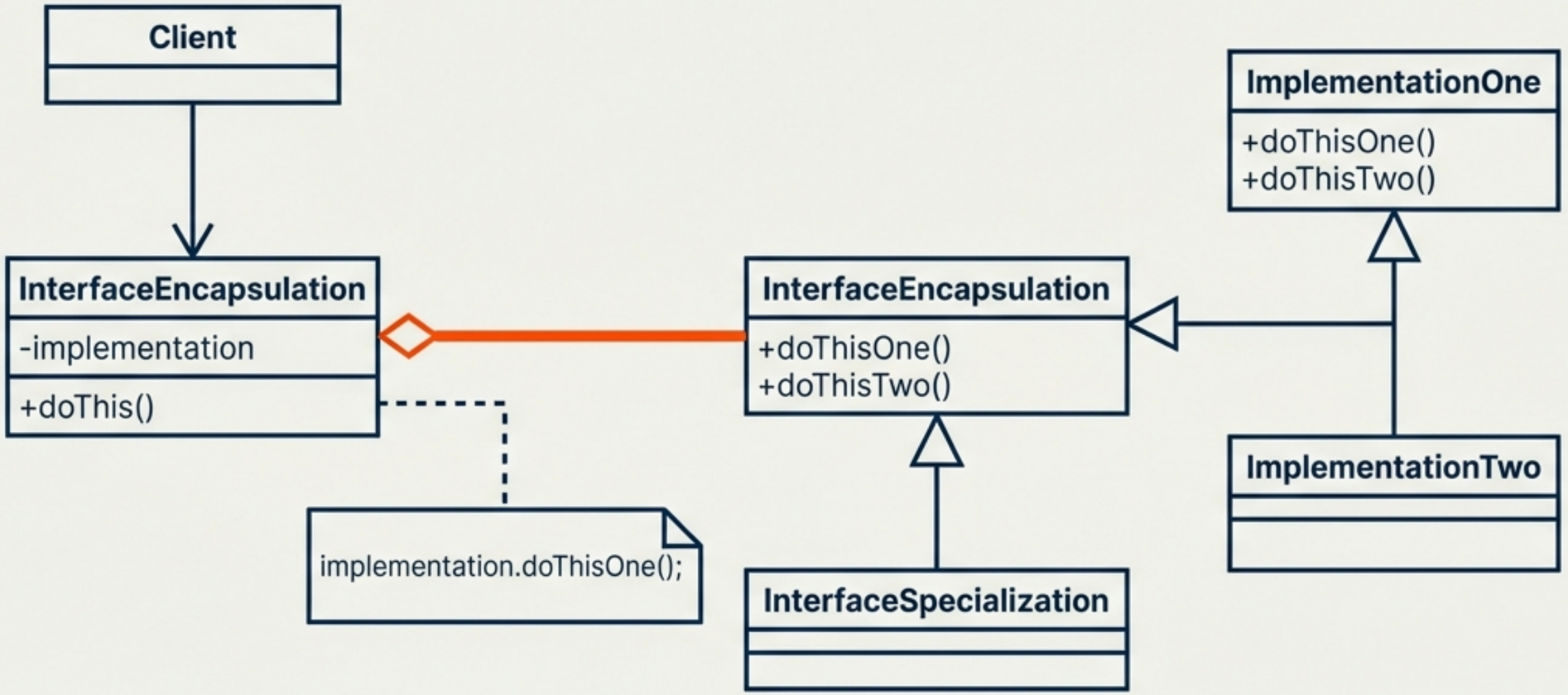
A Implementação (Plataforma)

O “trabalhador” de baixo nível (ex: API do Sistema Operativo). Realiza as operações primitivas.

A Abstração define 'o quê'. A Implementação define 'como'.

NotebookLM

Anatomia do Padrão



NotebookLM

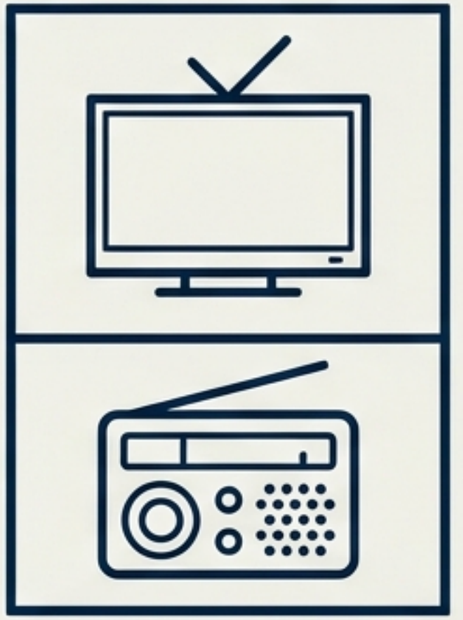
Metáfora: Comandos e Dispositivos

Abstração



RemoteControl
- togglePower()
- volumeUp()

Implementação



Interface Device
- enable()
- setVolume()

O mesmo **comando** funciona com qualquer **dispositivo** que siga o contrato Device.

NotebookLM

Mecânica de Implementação (Java)

```
public abstract class Shape {
    protected Color color; // A 'Ponte'

    // Injeção via Construtor
    public Shape(Color color) {
        this.color = color;
    }

    abstract public String draw();
}
```

Referência (Ponte)

Injeção de Dependência

O Cliente decide qual a implementação a ligar em tempo de execução (runtime).

NotebookLM

Estudo de Caso: Serviços de Conteúdo

Requisito: Acesso a serviços instanciados dinamicamente.



- **Problema**: Separar lógica de acesso da lógica de armazenamento.
- **Solução**: O Cliente interage com a Interface, que delega para o serviço instanciado.
- **Detalhe Técnico**: LinkedHashMap garante a ordem de inserção.

NotebookLM

Checklist de Aplicabilidade

- **Dividir Classes Monolíticas:** Quando uma classe tem **múltiplas variantes** de funcionalidade.
- **Dimensões Ortogonais:** Quando é necessário estender uma classe em **direções independentes** (Forma vs. Cor).
- **Troca em Runtime:** Quando precisa de mudar a implementação enquanto a aplicação está a correr.

NotebookLM

Análise de Trade-offs

Vantagens (Pros)

+ Princípio da **Responsabilidade** Única (SRP)
+ Princípio Aberto/Fechado
+ Princípio Aberto/Fechado (OCP)
+ Independência de Plataforma

Desvantagens (Cons)

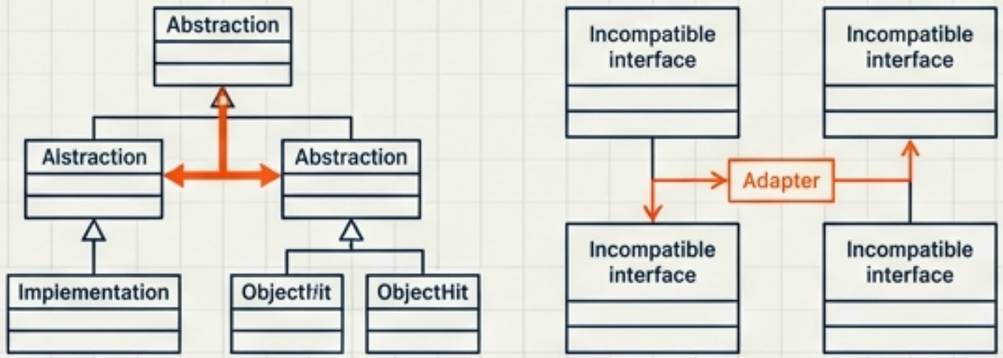
- Aumento da Complexidade (Mais Classes)
- Overhead para sistemas simples

NotebookLM

Relações com Outros Padrões

Bridge vs. Adapter

Bridge é desenhado UP-FRONT (antes do código).
Adapter é usado AFTER-THE-FACT (para compatibilidade).

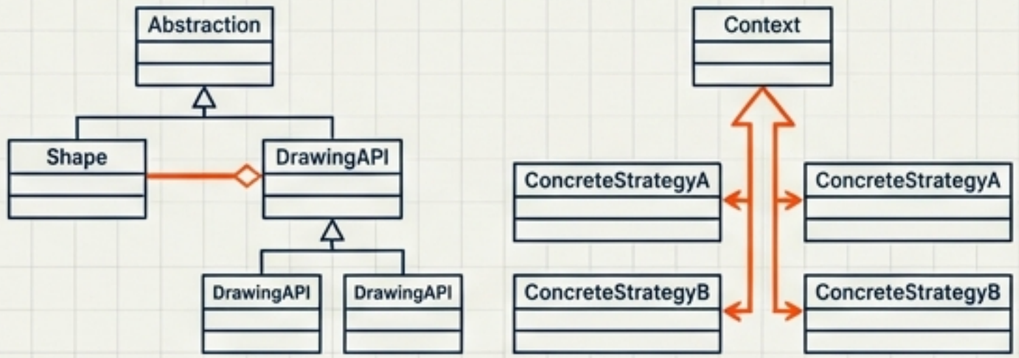


UP-FRONT DESIGN

AFTER-THE-FACT INTEGRATION

Bridge vs. Strategy

Ambos usam composição.
Bridge é ESTRUTURAL (como as classes são organizadas).
Strategy é COMPORTAMENTAL (como os algoritmos são trocados).

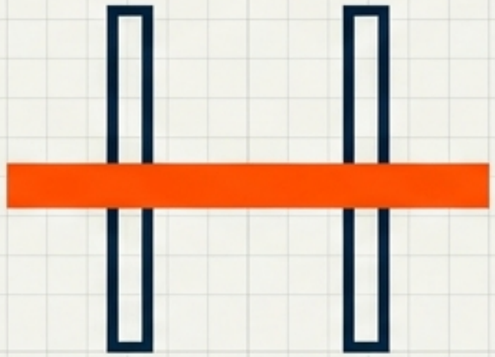


ESTRUTURAL

COMPORTAMENTAL

NotebookLM

Resumo Estratégico



“Prefira composição sobre herança para evitar o crescimento exponencial de classes.”

`Identificar Dimensões → Extrair Implementação → Ligar via Composição`

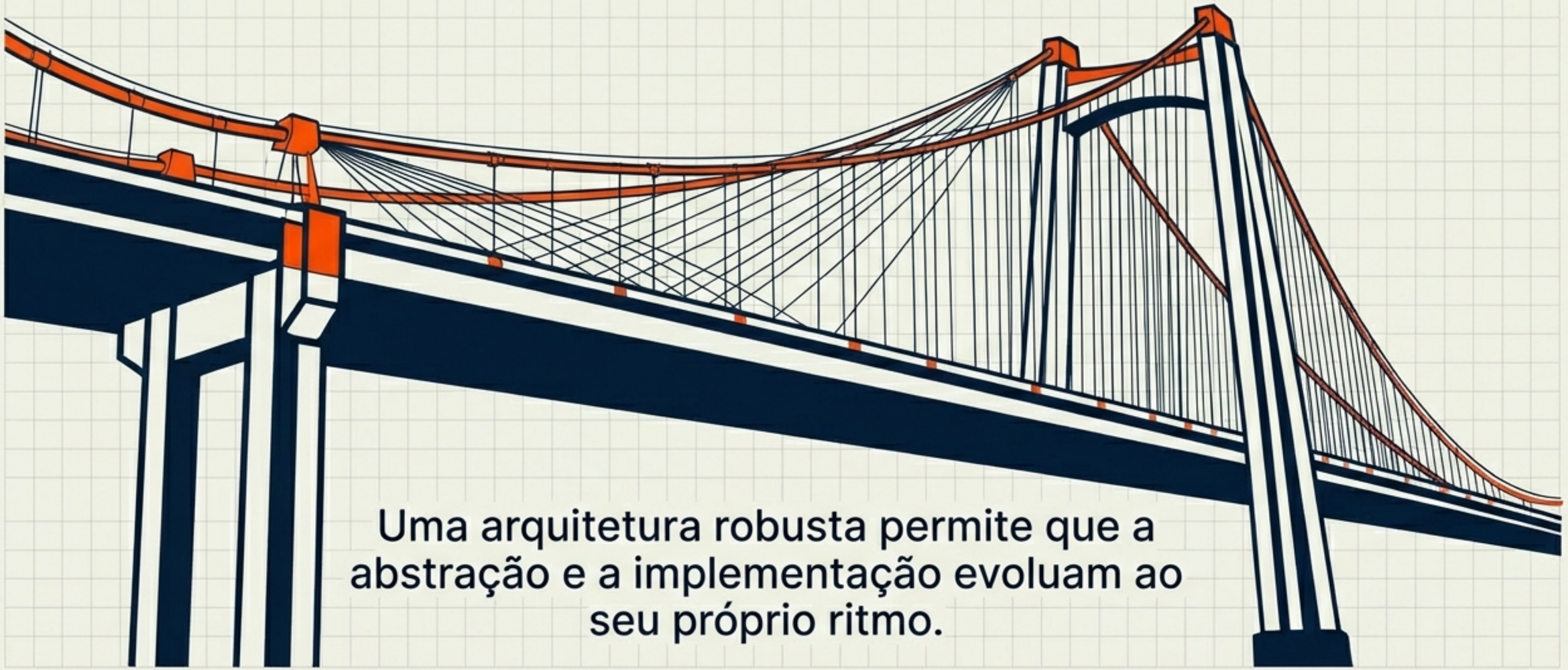
NotebookLM

Recursos Recomendados

- Refactoring Guru: "Dive Into Design Patterns" (Conceitos)
- Baeldung: "The Bridge Pattern in Java" (Implementação Técnica)
- Java Documentation: LinkedHashMap (Estruturas de Dados)
- DEV Community: “Understanding the Bridge Design Pattern”

NotebookLM

Construir Pontes, Não Monólitos.



Uma arquitetura robusta permite que a abstração e a implementação evoluam ao seu próprio ritmo.

NotebookLM